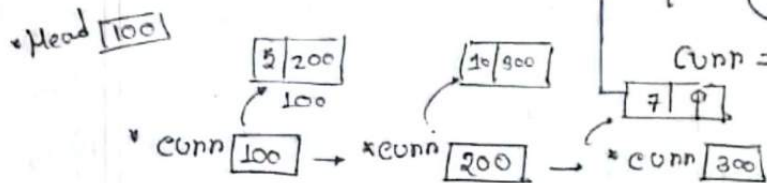


• print kora
prothekta value access kora
(traversal)

```
int main() {
    create();
    display();
}
```

```
void display() {
    struct node *curr;
    curr = head;
    while (curr != NULL) {
        printf("%d -> ", curr->data);
        curr = curr->next;
    }
}
```



• Insertion

- at beginning (creation en shomo)
- at end
- after a particular node

• Insertion at the beginning

steps

- * creating a new node
- * new node en next e → head
- * head → new node en address

```
void insert_at_beg() {
```

```
    struct node * newnode
```

```
    newnode = (struct node *) malloc (size of (struct node));
```

input
neya { scanf ("%d", & newnode->data);

```
    newnode->next = NULL;
```

```
    if (head == NULL) {
```

```
        head = newnode;
```

```
        tail = newnode;
    }
```

```
    else {
```

```
        newnode->next = head;
```

```
        head = newnode;
    }
```

• Insertion at the End

tail → end e insert

or puna list traversal karte hobe - time beshi lagbe

else {

```
    temp = head;
```

```
    while (temp->next != NULL) {
```

```
        temp = temp->next;
```

```
    temp->next = newnode;
```

```
    tail = newnode;
}
```

→ joto khon na null hoeshe



cw

03/03/26
Online
undirected graph

Tree

Graph

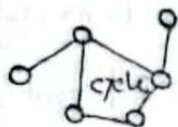
$$G = (V, E)$$

Node/vertex

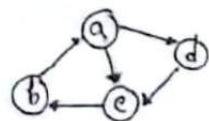
$$V = \{a, b, c, d, e\}$$

Edge

$$E = \{ac, ae, ad, de, ce, be\}$$

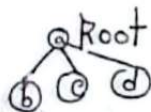


Directed graph



circuit/cycle

* connected graph (path a-e-h-e
* cycle free (vertices to v.)



Degree

1 to node en sathe joto gula edge connected
(undirected graph)

directed graph : in-degree ↓
out-degree ↑

Tree terminologies

- 1) Node
- 2) Edge
- 3) Root
- 4) Indegree / Outdegree
- 5) Parent Node
- 6) child node
- 7) sibling
- 8) leaf (last level) on (child n th thake) on (outdegree 0)
- 9) path

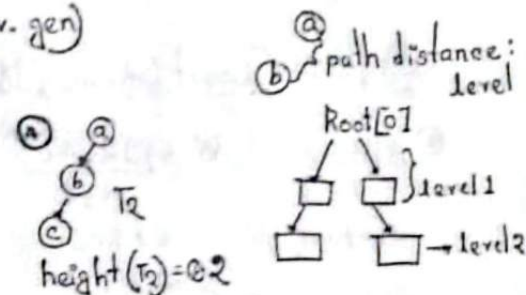
10) Descendent of a node (next gen. node)

11) Ancestor of a node (prev. gen)

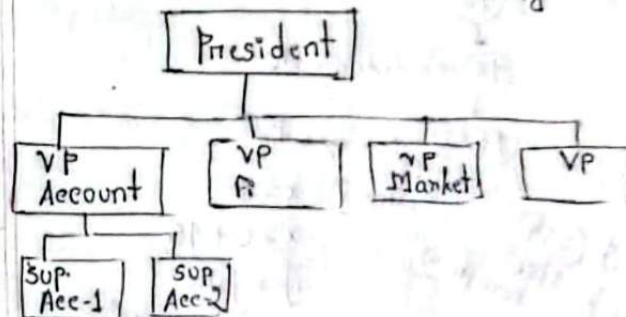
12) Level

13) Height

* Use of a tree



Hierarchical Relation * empty tree: * a tree Height = 0
height = -1



19/01/2026

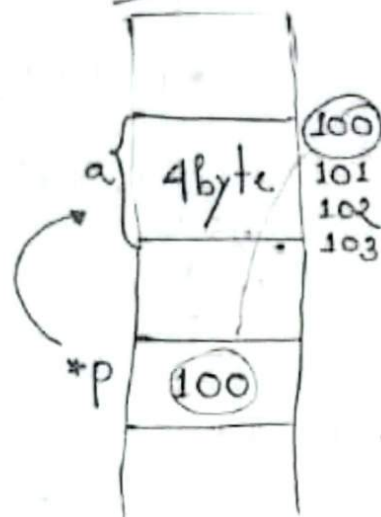
Pointer

variable that stores address of another variable

int a;

Memory

*** pointers : 8 byte usually



starting byte : address

size n-byte : (n-1)

int *p;

p = &a;

a = 20;

*p = 50; → a's value is 50

two different pointers can point the same variable

$\left\{ \begin{array}{l} p = \&i; \\ q = \&j; \end{array} \right. \quad \begin{array}{l} *p \rightarrow i \\ *q \rightarrow j \end{array}$

*q = *p; → p jake point konche tar value, q jake point konche tar value te assign kora hoeche

alternative j = i;

local variable

Uses of Pointer

call by value

swap

kaj hobe na swap hobe na

int main()

int x = 5, y = 10;

printf("x = %d, y = %d", x, y);

swap(x, y);

printf("x = %d, y = %d", x, y);

(x, y) not same address

void swap(int x, int y)

{ int temp;

temp = x;

x = y;

y = temp;

call by address/reference

$x = 5, y = 10$
`swap(&x, &y);`

$x \rightarrow [5, 10] \text{ } 100$
 $y \rightarrow [10, 5] \text{ } 200$

10 hobe
 5 hobe

`void swap (int *p, int *q)`

`int temp;`

`temp = *p;`

`*p = *q;`

`*q = temp;`

$*p \text{ } [100]$

$*q \text{ } [200]$

$\text{temp} \text{ } [5]$

20 byte lagbe memorye
 (5x4) consecutive

Pointer - Array

`int main() {`

`int A[5] = {1, 2, 3, 4, 5}; *p, i;`

`p = &A[0];` or `p = A;` $A[0]$ ke refer kore

`printf("%d", *p);`

`p++;` $\rightarrow$ pointer variable ke point konbe

`printf("%d", *(p+1));` $p+1$
 $\rightarrow 108 \text{ } 100 + 1 \times 4$
 p change hoy nai 104

`printf("%d", *p);` (now p is pointing $A[1]$)
 $\rightarrow 104 \rightarrow A[1]$

	1	100
A[0]		
	2	104
A[1]		
	3	108
A[2]		
	4	112
A[3]		
	5	116
A[4]		
*p	100 $\rightarrow$	104

output screen

1 2 3 2

```

else {
    tail → next = newnode;
    newnode → prev = tail;
    tail = newnode;
}

```

• Inserting at any position

```

void insert_pos() {
    create();
    if (head == NULL) { head = tail = newnode; }
    else {
        temp = head;
        for (i = 1; i < pos; i++) {
            temp1 = temp;
            temp = temp → next;
        }
        temp1 → next = newnode;
        temp → prev = newnode;
        newnode → next = temp;
        newnode → prev = temp1;
    }
}

```

alternative

```

else {
    temp = head;
    for (i = 1; i < pos - 1; i++) {
        temp = temp → next;
    }
    newnode → prev = temp;
    newnode → next = temp → next;
    temp → next → prev = newnode;
    temp → next = newnode;
}

```

Deleting

• Deletion at Beginning

```

void deleteBeg() {
    temp = head;
    head = head → next;
    head → prev = NULL;
    free(temp);
}

```

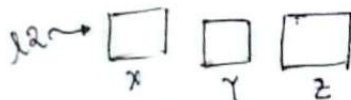
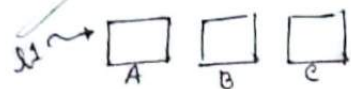
• Deletion at the End

```
void deleteEnd() {
    temp = tail;
    tail = tail->prev;
    tail->next = NULL;
    free(temp);
}
```

• Deletion at any position

```
temp1->next = temp->next;
temp->next->prev = temp1;
free(temp);
```

Reverse
node count



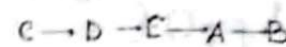
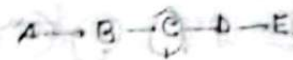
tail1 = next = head2

```
temp = head;
while (temp->next != NULL) {
    x = temp->prev;
    y = temp->next;
    z = x;
    x = y;
    y = z;
    head = if (temp->prev == NULL)
            temp = head;
    if (temp->next == NULL)
        temp = tail;
}
```

temp = head;

```
for (j=1, j<=n, j++) {
    head = head->next;
}
```

```
for (j=1, j<=n, j++) {
    tail = tail->next;
    tail->next = NULL;
}
```



j=1, head=B

j=2, head=C

j=1

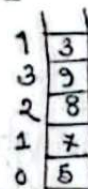
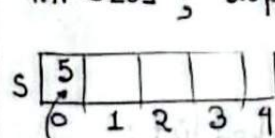
10/02/2026

Stack:

Mechanism: Last in first out (LIFO)

Insertion: push
insertion and deletion from top

int S[5]; top = -1



i) push(5): top++

ii) push(7)

top = -1, 0, 1, 0

void push(int item)

```
{ if (top + 1 == size) -> size of stack overflow
  else -> top++ }
```

① { top = -1
top = 0
S[0] = 5

② { top = 0
top = 1
S[1] = 7

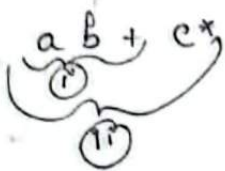
③ { top = 1 -> top = 2
S[2] = 8

④ { top = 2 -> top = 3
S[3] = 9

⑤ { top = 3 -> top = 4
S[4] = 3

⑥ { top = 4 -> 5
Stack overflow

$$= ab + c \times$$



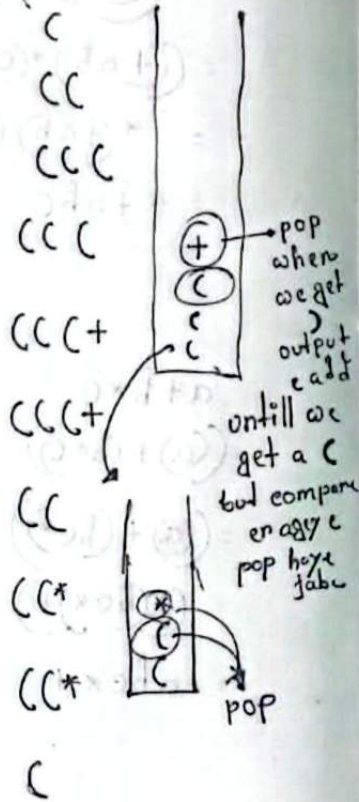
$$4) ((A + B) * C) - ((D + E) / F)$$

Step	Token	Postfix
1	(	
2	(	
3	(	
4	A	A
5	+	A
6	B	AB
7	)	AB+
8	*	AB+
9	C	AB+C
10	)	AB+C*

in: $a + b$
pos: $ab +$
pre: $+ab$

→ push

stack

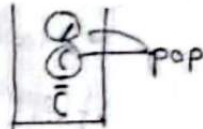
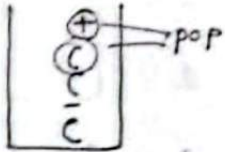


untill we
get a C
but compare
energy
pop hoxe
jabe

prof

Step

11	-	$AB + C^*$	$(-$
12-13	((	$AB + C^*$	$(-(($
14	D	$AB + C^*D$	$(-CC$
15	+	$AB + C^*D$	$(-CC+$
16	E	$AB + C^*DE$	$(-CC+$
17	)	$AB + C^*DE +$	$(-($
18	/	$AB + C^*DE +$	$(-(/$
19	F	$AB + C^*DE + F$	$(-(/$
20	)	$AB + C^*DE + F/$	$(-$
21	)	$AB + C^*DE + F/-$	



$$(A+B)*C - (D+E)/F$$

- 1 ((
- 2 A A
- 3 +
- Stack-top check
if (opening bracket on Null → push)
if (operator: priority check)



- 4 B AB (+

- 5) AB+

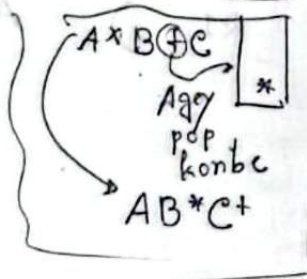
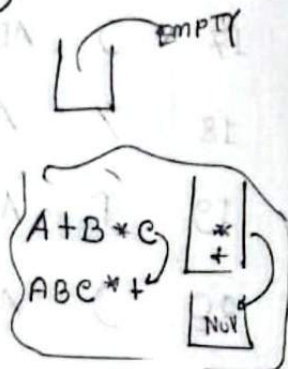
- 6 * AB+ *

- 7 C AB+C *

- 8 - AB+C* -

- 9 (AB+C*~~D~~ - (
- 10 D AB+C*D - (
- 11 + AB+C*D - (+

- 12 E AB+C*DE - (+

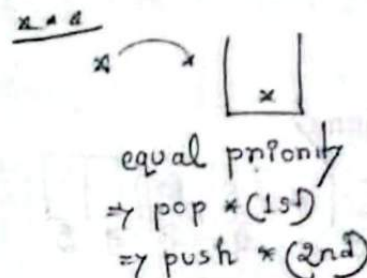


- 13) AB+C*DE+ -

- 14 / AB+C*DE+ -/

- 15 F AB+C*DE+F -/

- 16 \$ AB+C*DE+F/-



$$AB + C * DE + F / -$$

$$2 \ 3 \oplus 4 \times 5 \ 6 + 7 / -$$

pop immediate two operand
and apply that operator

$$i) 2+3 = 5 \rightarrow \text{push}$$

$$ii) 5 \times 4 = 20$$

$$iii) 20 + 5 = 25$$

$$iv) 25 / 2 = 12.5$$

$$\frac{1.5}{20}$$

$$\frac{18.5}{20}$$

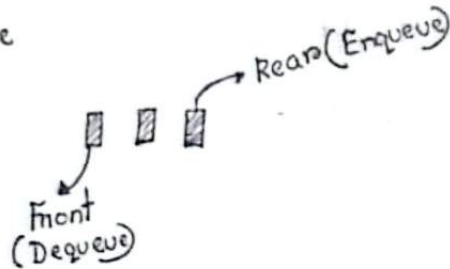
18/03/2026

Queue :

Mechanism : first in first out (FIFO)

i) Enqueue

ii) Dequeue



#Queue implementation using Array

```
int Q[size];
```

```
int f=-1, r=-1;
```

enqueue(5):

$f=0, r=0, Q[0]=5$ (As the queue was empty)
 $\hookrightarrow f=-1, r=-1$

enqueue(7):

$f=0, r=1, Q[r]=Q[1]=7$

enqueue(8): $f=0, r=2$

enqueue(10): $r=3$

enqueue(2): $r=4$

enqueue(7): $r=5$

enqueue(20): Q overflow $\rightarrow$ if $r \geq \text{size}$

5	7	8	10		
0	1	2	3	4	5

```
void enqueue (item) {
    overflow ← if (r+1 == size) { Q full; }
    Empty ← else if ((f == -1 && r == -1) { f++; r++; Q[r] = item; }
    else { r++; Q[r] = item; }
```

```
int Q[size], f=-1, r=-1;
```

Dequeue

- i) deq(): $Q[f] \rightarrow "5", f++$ $f=1$
- ii) deq(): $Q[f] \dots f++$ $f=2$
- iii) deq(): $Q[f] \dots f++$ $f=3$
- iv) deq(): $Q[f] \dots f++$ $f=4$
- v) deq(): $Q[f] \dots f++$ $f=5$

$\hookrightarrow f=-1, r=-1$

vi) deq(): Q Empty

```
void dequeue() {
    Empty → if (f == -1 && r == -1) { Q Empty; }
    last element → else if (f == r) { print Q[f]; f=-1; r=-1; }
    else { print Q[f]; f++; }
```

26/02/2026

Recursion

$$f(4) = 5 \times (4-3) / 4 \rightarrow \text{final representation}$$

Recursive function: $f(x) = 2f(x-1) + x^2$ | Recursive Case
 $f(x-1)$ er value jana lagbe

$$f(0) = 0 \quad (\text{Base Case})$$

$$f(4) = 2f(3) + 4^2 \quad f(4) = 58$$

$$f(3) = 2f(2) + 3^2 \quad f(3) = 21$$

$$f(2) = 2f(1) + 2^2 \quad f(2) = 6$$

$$f(1) = 2f(0) + 1^2$$

$$f(1) = 1 \quad (\text{Now we can do back calculation})$$

Example

Factorial $N! = N * (N-1)!$

$$L_5 = 5 * 4 * 3 * 2 * 1$$

$$= 5 * L_4$$

$$= 5 * 4 * 3 * 2 * 1$$

$$L_5 = 5 * 4 * 3 * 2 * 1 * 1$$

Base Case:

$$1! = 1, 0! = 1$$

int factorial (int n) {

int t;

→ if (n <= 1) { t = 1; }

→ else { t = n * factorial(n-1); }

return t;

int main() {

input n;

marks = factorial(n);

print f;

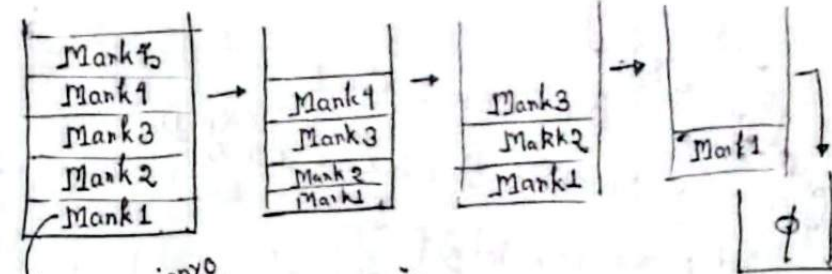
Mark 2 fact(4): (n=4, t=4 * fact(3))

Mark 3 fact(3): n=3, t=3 * fact(2); return t

Mark 4 fact(2): n=2, t=2 * fact(1); return t

(Base case) Mark 5 fact(1): n=1, t=1; return t;

use of stack



fact(4) enjonyo shb variable (n, t) as fact(4) ekhono terminate hoy nai, tai shb variable exist konbe, return ni hoya porjonto stacke store hobe

Non-Terminating Recursion

```
int Bad(int n) {
    if (n == 0) { return 0; }
    else { return (Bad(x/3+1) + x-1); } }
```

- BAD(4) : return BAD(2) + 3
- BAD(2) : return BAD(1) + 1
- BAD(1) : return BAD(0) + 0
- Base case e reach konbe nh. $BAD(0) =$
- stack overflow konbe (after a certain time)
- (memory allocate hoeebe nh)

Base Case : $x^0 = 1$

Recursive Case : $x^n = x \times x^{(n-1)}$

```
int power(int x, int n) {
    int res;
    if (n == 0) { return 1; }
    else { res = x * power(x, n-1); }
    return res; }
```

Fibonacci

```
long fibonacci(long n) {
    if (n == 0 || n == 1) { return n; }
    else { return fibonacci(n-1) + fibonacci(n-2); } }
```

$f(x) = f(n-1) + f(n-2)$ (Recursive Case)

$n=0 \rightarrow 0, n=1 \rightarrow 1$ (Base Case)

0 1 1 2 3 5
1 2 3 4 5 6

Overhead

Recursion vs. Iteration

Repetition : i) It $\rightarrow$ explicit loop
ii) Re $\rightarrow$ repeated function call

Termination : i) It $\rightarrow$ loop condition falls
ii) Re $\rightarrow$ base case recognized

Termin of
H

Balance of performance & software engineering
between time complexity, space complexity (Iteration) and program is easy to read (Recursion)

Section-2 (Quiz Ques)

02/03/2026

① struct node {
 int data;
 struct node *next;
 *h, *b, *z; → 12 (tail)
 new head
 (12) temp = b
 while (temp != NULL) {
 create newnode; (newnode → data = temp → data)
 if (b == NULL) { h = z = newnode; }
 else { z → next = newnode;
 z = newnode; }
 temp = temp → next; }
 }
 2 5 9 8

② sorted LL
 insert data in order
 pos = 0
 temp != NULL {
 if (temp → data < key) break;
 else { pos++; }
 temp = temp → next; }
 }
 key = 7
 c++
 temp = temp → next

while (temp != NULL) {
 if (temp → data < key) { temp1 = temp;
 temp = temp → next; }
 else { temp1 → next = newnode;
 temp → prev = newnode;
 newnode → prev = temp1;
 newnode → next = temp; }
 if (temp == NULL) { temp1 → next = newnode;
 newnode → prev = temp1; }
}

Section-1 (Quiz)

① for (i = e; i > n; i--) {
 temp = temp → next;
 printf (temp → data);
}

A B C D E
 3

② while (temp != NULL) {
 temp1 = temp → next;
 while (temp1 != NULL) {
 if (temp → data + temp1 → data == x) { print;
 break; }
 else { temp1 = temp1 → next; }
 temp = temp → next; }
}

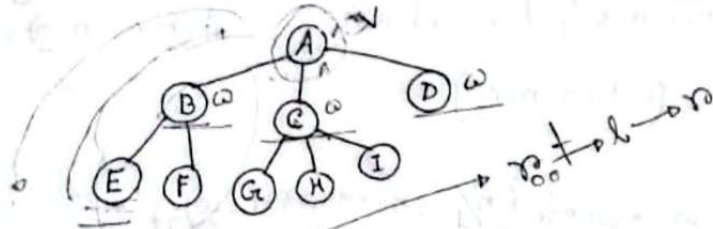
2 5 7 10
 10 + 2 = 12
 but 2
 thakan
 chance
 nat

11/03/2026

Online

Tree traversal

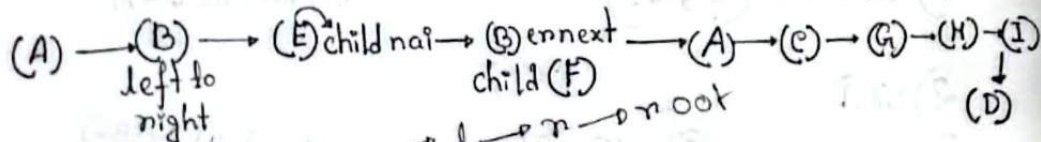
(Non linear data structure)



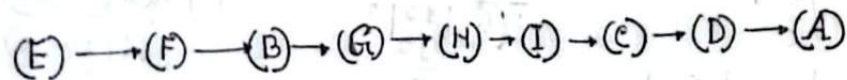
i) Pre order: visit parent before child

- here we start with root (A)

- then either we start from left to right or right to left



ii) Post order: visit parent after child



(A) the shuru but (A) ke access konh-hoy nai

Algorithm of Pre Order:

```

preOrder(V) {
    visit(V)
    for each child w of V
        preOrder(w)
}

```

i) preOrder(A): Print A(visit)

for
preOrder(B), preOrder(C), preOrder(D)

ii) preOrder(B): visit B

pre(E), pre(F)

iii) pre(E): child nai, recursively new function call, hobe nh return to pre(F)

iv) pre(F): visit F, program will return to (C)

v) pre(C): visit C

pre(G), pre(H), pre(I)

;

vi) pre(D): return pre(A)

Algorithm of Post Order:

```

postOrder(V) {

```

i) for each child w of V
postOrder(w)

visit V



i) post(A) : post(B), post(C), post(D)

ii) post(B) : post(E), post(F)

iii) post(E) : visit E, return to post(F)

iv) post(F) : visit F, return to post(B) visit(B)

v) post(C) : post(G), post(H), post(I)

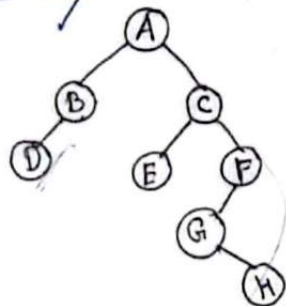
vi) post(G) : visit G, return

vii) post(H) : visit H, return

viii) post(I) : visit I, return

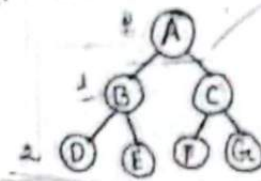
ix) post(D) : visit(D) return to visit A

Binary Tree



child not more than 2
left child, right child

Complete Binary Tree



maximum possible node that is possible

• Maximum number of nodes in level $l = 2^l$

• for height h leaf = 2^h

n : number of nodes

h : height of this tree

$$n \leq 2^{h+1} - 1$$

$$\Rightarrow 2^{h+1} - 1 \geq n$$

$$\Rightarrow 2^{h+1} \geq n+1$$

$$\Rightarrow h+1 \geq \log_2(n+1)$$

$$h \geq \log_2(n+1) - 1$$

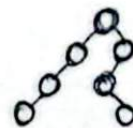
$$h_{min} = \log_2(n+1) - 1$$

$$h = \log_2(7+1) - 1$$

$$= 3 - 1$$

$$= 2$$

2 en beshi hole pane



$$1st\ level: 2^0 = 1$$

$$2nd: 2^1 = 2$$

$$3rd: 2^2 = 4$$

$$h_{max} = n - 1$$



h min for 10 nodes :

$$h_{min} = \log_2(n+1) - 1$$

$$h_{min} = \lceil \log_2(11) \rceil - 1$$

$$h_{min} = \lceil 3.47 \rceil - 1$$

$$h_{min} = 3$$

Binary tree traversal

i) Pre Order (F, D, B, A, C, E, J, G, I, K)

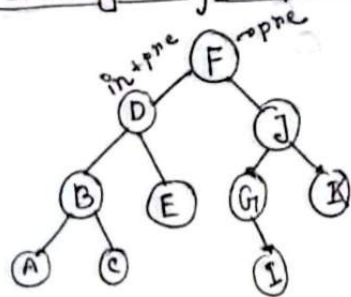
ii) Post order (A, C, B, E, D, I, G, K, J, F)

iii) In order

l → root → right

(A, B, C, D, E, F, G, I, J, K)

drawing tree from pre-order & in-order:

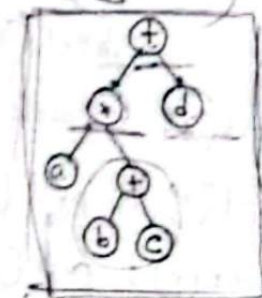
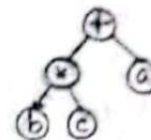


Expression trees

a+b

operand → leaf

operator → root



Infix traversal

Output →

infix (in(+); print "("; in(*); print "+"; in(d); print ")")

((in(*) : print "("; in(a); print "*"; in(+); print ")")

((a*b in(a) : print "a"

((a*(in(+): print "(", in(b); print "+"; in(c); print ")")

((a*(b in(b) : print "b"

Postfix Traversal

$a * (b + c) + d$

$abc + * d +$

post(+): [post(*); post(d); print "+"]

post(*): post(a); post(+); print "*"

post(a): post(φ); post(φ); print "a"

post(φ): tree empty → return

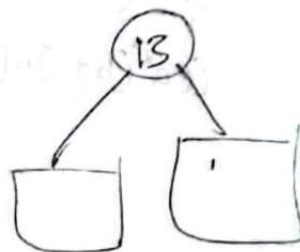
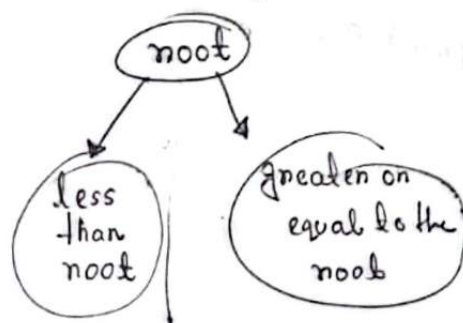
post(φ): return

a

post(+): post(b), post(c), print "+"

$abc + * d +$

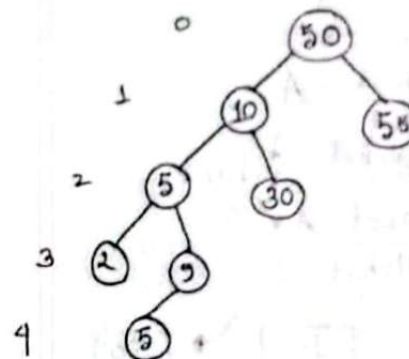
Binary Search Tree (BST)



Inserting

50, 10, 5, 30, 35, 2, 9, 5

Root



height = 4

height_{max} = n-1

h_{min} = $\lceil \log_2(n+1) \rceil - 1$

= $\lceil \log_2 9 \rceil - 1$

= 3

Implementation

struct BT-node {

int data;

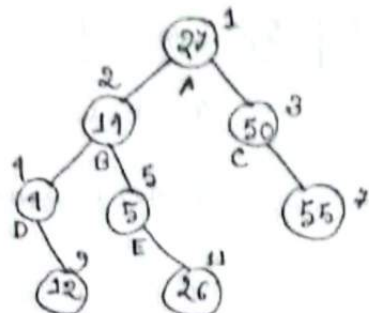
struct BT-node *left, *right; *root = NULL;

*root = NULL

$a * (b + c) + d$

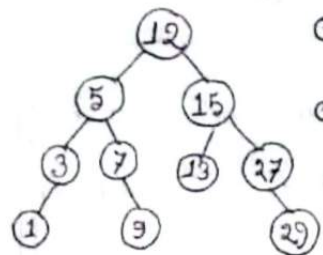
Array

$\text{parent} = i$
 $\text{left child} = 2i$
 $\text{right child} = 2i + 1$



27	14	50	10	25	X	55	X	12	X	26
1	2	3	4	5	6	7	8	9	10	11

Delete a node from BST



case: 1) leaf node (node with 0 child)

case: 2) node with one child

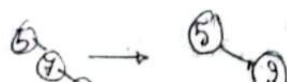
case: 3) node with two children

case: 1) leaf node

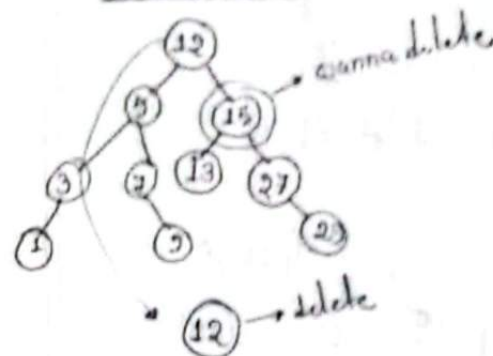
- free kora
- parent node en left or right node en position kenu kora

case: 2)

- je node delete konbo tan parent and child add



case: 3



je node delete korte chachi
tan ^{right} subtree-er min-value node

or,
left subtree
max-value node.

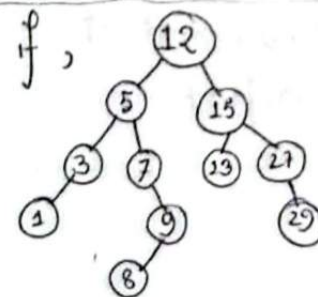
if wanna delete 12:

right subtree-er min-valued node: 13

delete 12

12 er space e 13

13 free kora (case 1 or case 2)



9 - new root

but eiban 9 en child 8 ke
9 en sathe attach kora

Right
subtree
min

Left
subtree
max value

min -> max -> 9

12 er space e 9

case 1

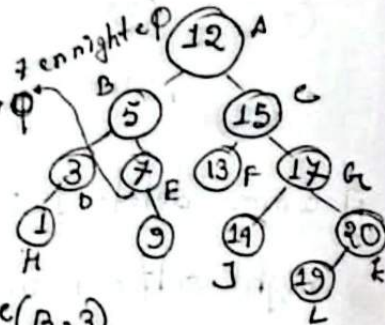
delete(A, 9): root → left = delete(B, 9)

delete(B, 9): root → right = delete(E, 9)

delete(E, 9): root → right = delete(I, 9)

delete(I, 9): free(root)

root = Q;
return root;



case 2

delete(A, 3): root → left = delete(B, 3)

delete(B, 3): root → left = delete(D, 3)

delete(D, 3):

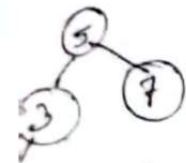
if (root → right == NULL)

struct node *temp = root; D

root = root → left = H

free(temp); D

return(root);



case 3

delete(A, 15): root → right = delete(C, 15)

delete(C, 15): temp = findMidn(root → right) G
temp = G
root → data = temp → data
root → right = delete(root → right, G)

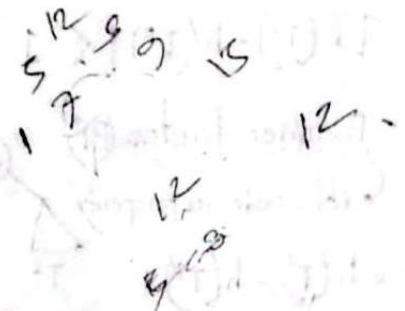
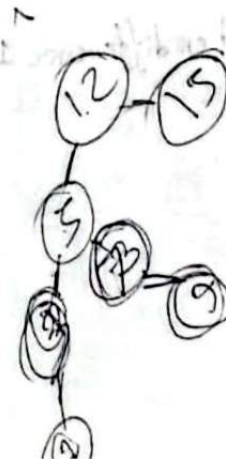
temp → data = 17

delete(G, 17): temp = root → G

root = root → right = K

free(temp);

return root; K



balanced tree

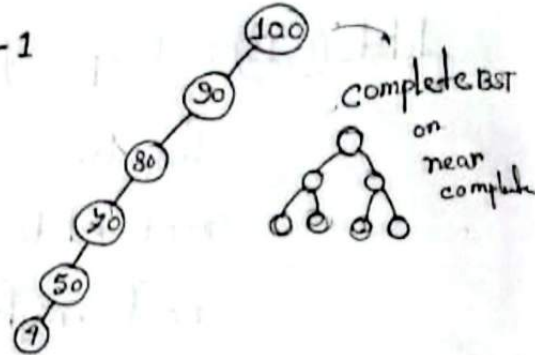
06/04/2026

AVL tree (Adelson velgik and landis)

$$h_{min} = \log_2(n+1) - 1$$

$$h_{max} = n - 1$$

n = no. of nodes



near complete binary tree



- last level 2nd node naothakle parent (only for last level)
- last level is left aligned

AVL tree

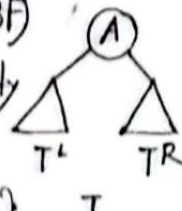
- BST
- left subtree & right subtree height difference 1 or less

$$|h(T^L) - h(T^R)| \leq 1$$

Balance factor (BF)

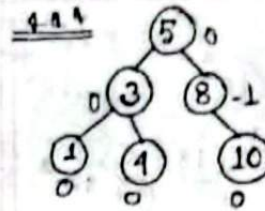
→ ekta node er property

$$BF = h(T^L) - h(T^R), \quad \{0, 1, -1\}$$



height o theke calculate
Empty tree -> height = -1

prontek ta node er BF check

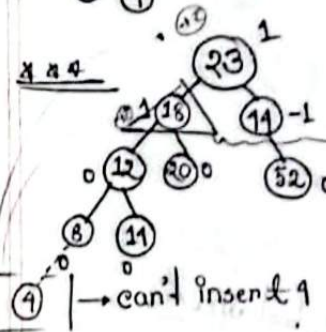
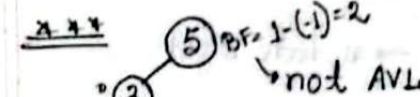


$$5 \rightarrow BF = 2 - 2 = 0$$

$$3 \rightarrow BF = 0 - 0 = 0$$

$$1 \rightarrow BF = -1 - (-1) = 0$$

$$8 \rightarrow BF = -1 - 0 = -1$$



considering that -
Imbalance at node

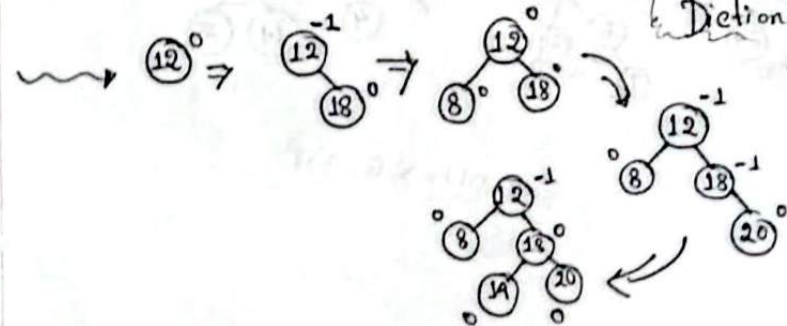
first imbalanced tree parent

file e data
↓
store
↓
any LL, BST

height balanced
nh hole BST-er
advantages paawa
facche nh

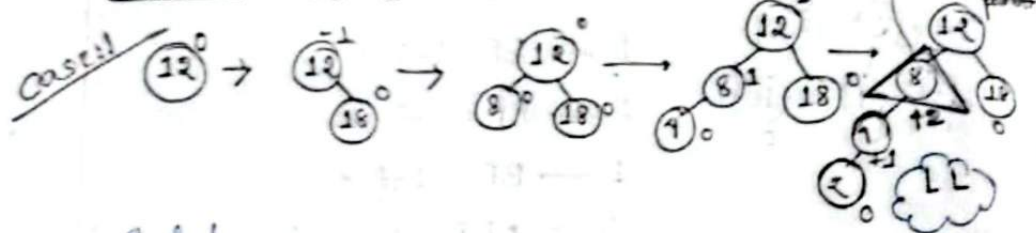
Construction

12, 18, 8, 20, 14



Dictionary → example

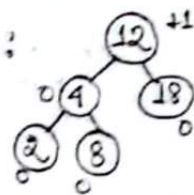
Case 1: 12, 18, 8, 2



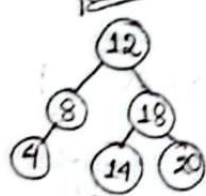
Imbalance:

- i) Left-Left (LL) : +2, +1 $\rightarrow$ rotate right
- ii) Right-Right (RR) : -2, -1
- iii) Left-Right (LR) : +2, -1
- iv) Right-Left (RL) : -2, +1

Rebalance:

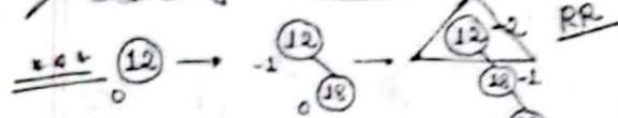


Rebalance:

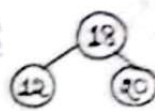


SL, DLL, S, Q, BST

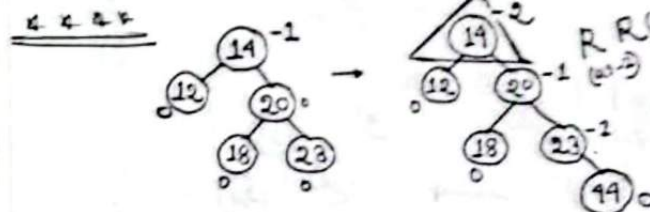
Case 02: Right-Right imbalance



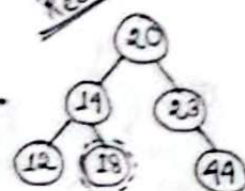
Rebalance:



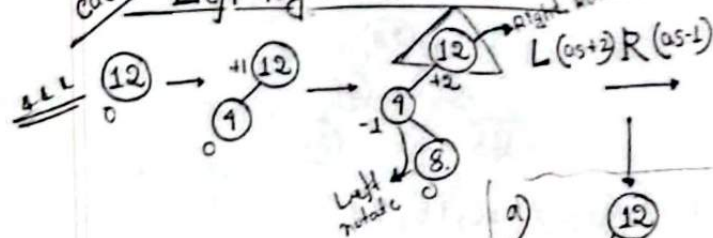
Case 03:



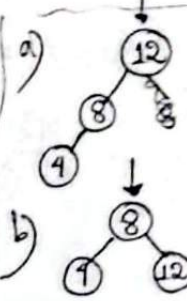
Rebalance:



Case 04: Left-Right imbalance (Right of Left)



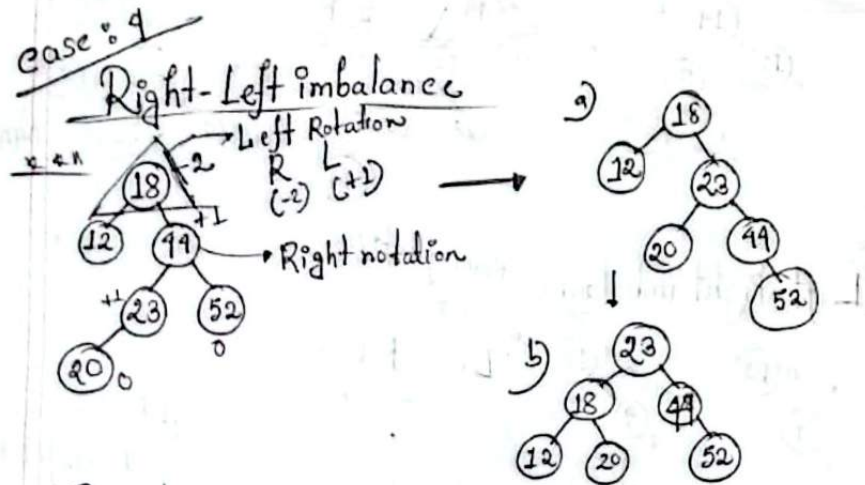
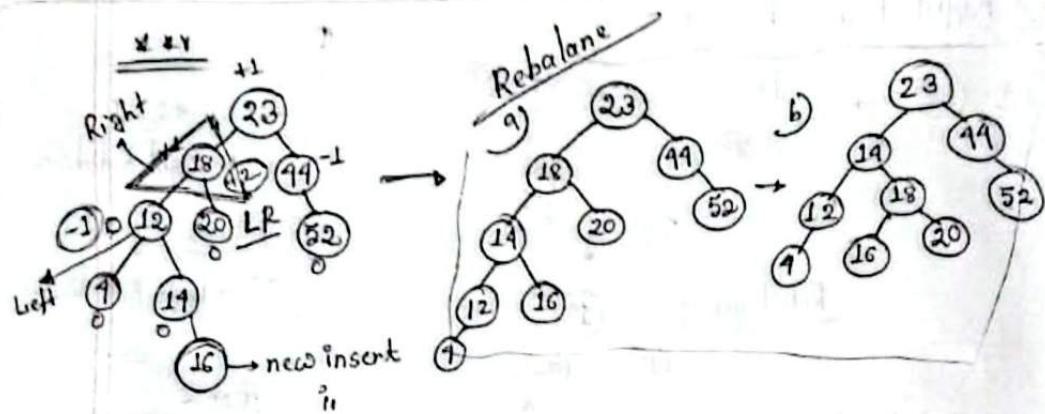
Rebalance:



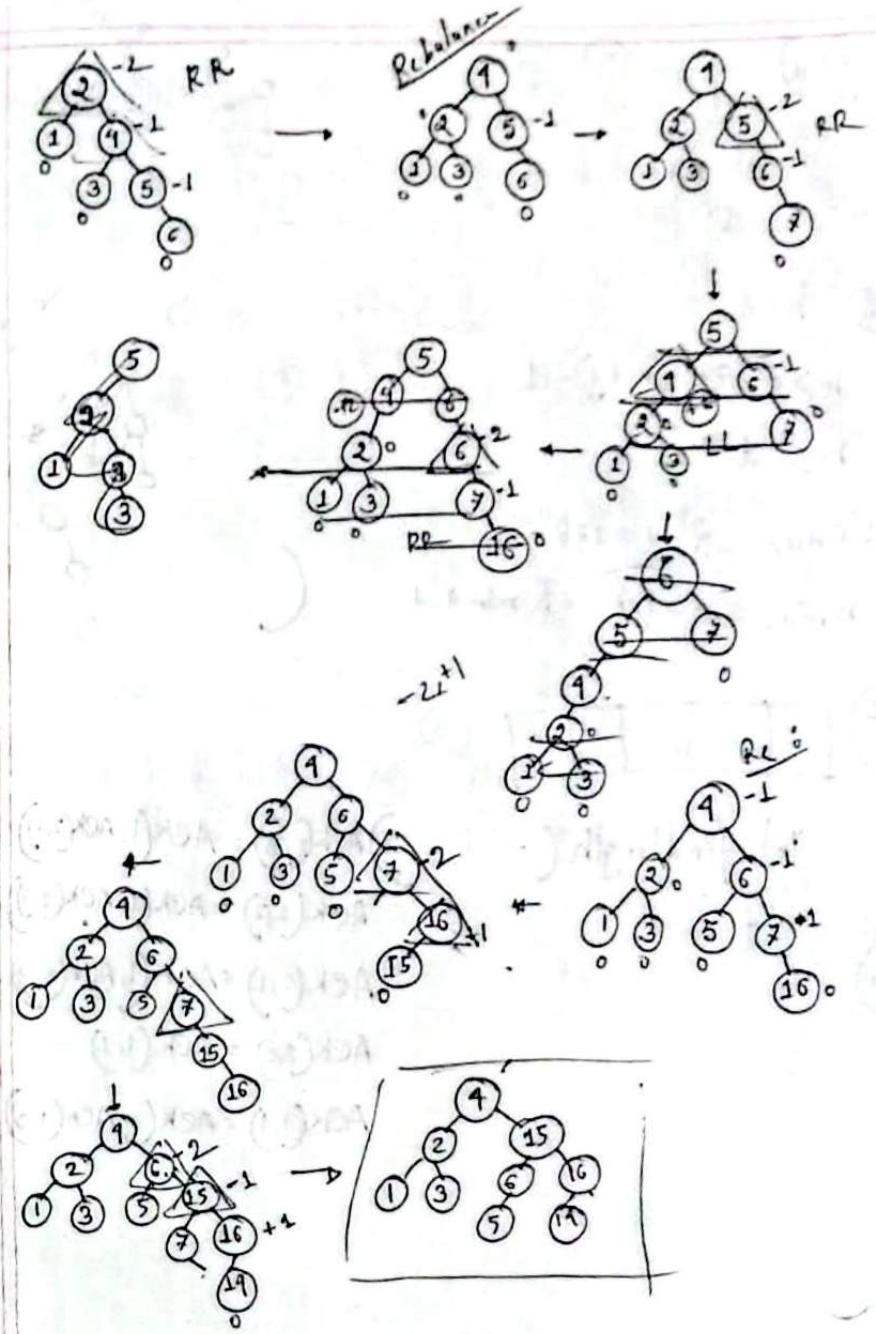
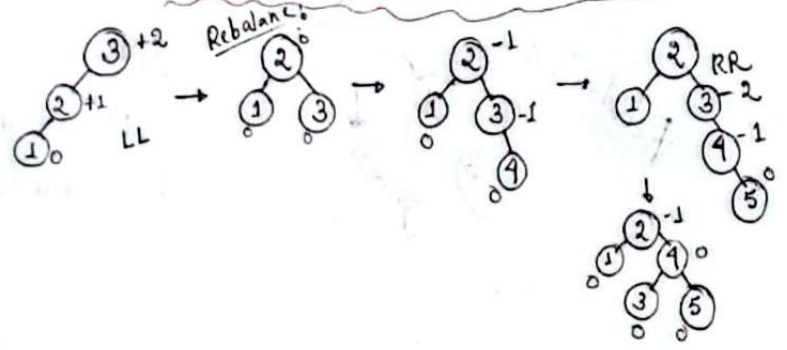
- 3) LR: +2, -1
- a) -1 $\rightarrow$ Left Rotation
- b) +2 $\rightarrow$ Right Rotation

1) RL: -2, +1

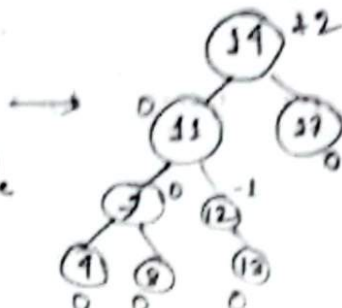
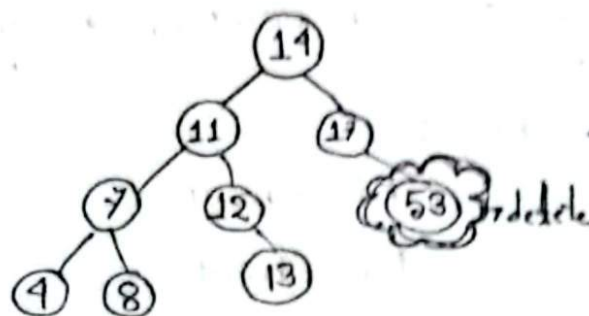
- a) +1 $\rightarrow$ Right Rotation
- b) -2 $\rightarrow$ Left Rotation



Example : 3, 2, 1, 4, 5, 6, 7, 16, 15, 19



AVL Deletion



12, 0

unique case

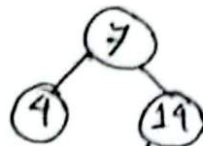
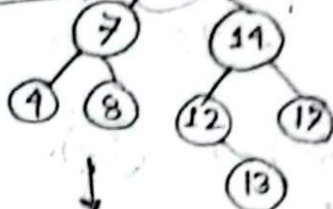
either consider

LL

or

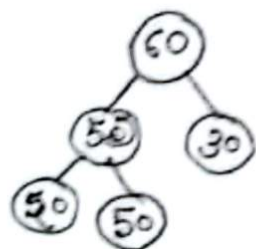
LR

Rebalance: 14 -> delete

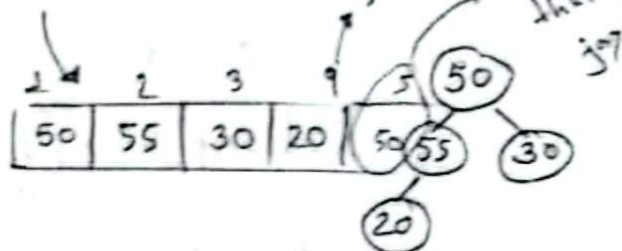


Max Heap

20, 50, 30, 60, 55



1	2	3	4	5
60	55	30	20	50



count = 5

del = Data[1] = 60

data[1] = data[count]

count -- ;

4

Reheap down (1, count)

pos = 1, left child = 2, right = 3

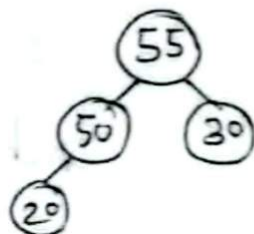
largest = pos = 1

largest = 2

swap(largest, position) → swap(2, 1)

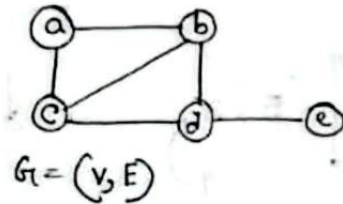
Reheap down(largest, count) → Reheap down(2, 4)

→ largest = 2 pos = 2, left = 4 right = 5



Graph

$V = \{a, b, c, d, e\}$
 $E = \{ac, ba, \dots\}$



i) Vertex :

ii) Edge :

iii) Adjacent : (edge diye connected)

iv) Path : (a to d) : $P_1 = acd$ or (ae, cd)
 path en length 2

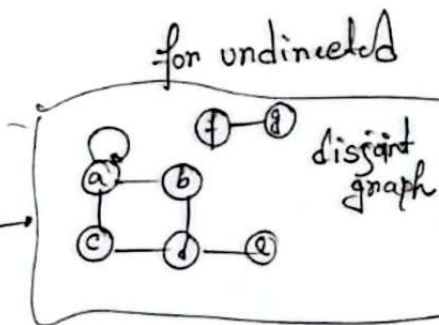
v) Cycle : $P_2 = abcd$ (ab, bc, cd)
 cycle en minimum length 3 (min 3 tanak lap)
 closed path, starting - ending vertex same
 circuit

vi) loop : (a)

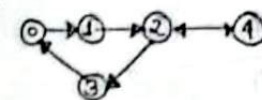
vii) Directed

viii) Undirected

ix) Connected graph

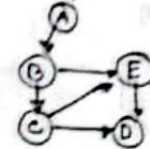


for directed graph



strongly connected

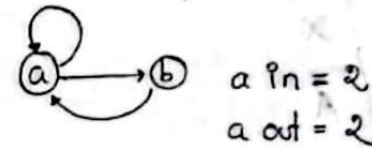
weakly connected



Degree

in out

ekta node en sathe fotogula edge connected
 full graph en degree : max degree



DFT

step	DFT	stack
0		A
1	A	X
2	AX	G, H
3	AXH	G, E, P
4	AXHP	G, E
5	AXHPE	G, M, Y
6	AXHPEY	G, M
7	AXHPEYM	G, J
8	AXHPEYMJ	G

HPop
 H en neighbour
 G en neighbour but
 already in last stack

9 AXHPEYMJG
 10

20/09/2024

● Structure

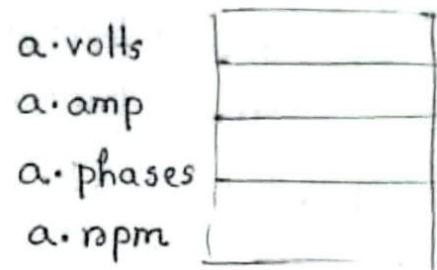
→ user defined data type

```
struct motor {
    float volts;
    float amp;
    int phases;
}; float rpm;
```

```
struct motor a;
a.volts = 220.0;
```

```
struct motor b[5];
```

memory



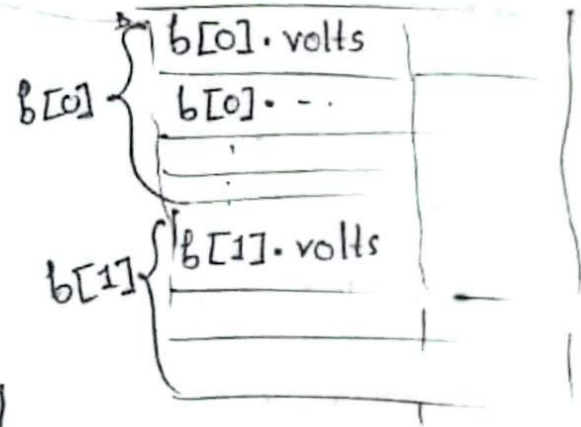
● pointing structure variable

```
struct motor a[5], *m;
```

```
m = &a[0];
```

(m++) → 16 byte jump
advance a[1]

(a[0] → a[0].volts, a[0].amp, ...)
1 byte 1 byte 16 byte



```
struct motor a[5], *m;
```

```
m = &a[0];
```

```
a[0].volts = 200;
```

```
(m++) * (m).amp = 20, or, m-amp = 20;
```

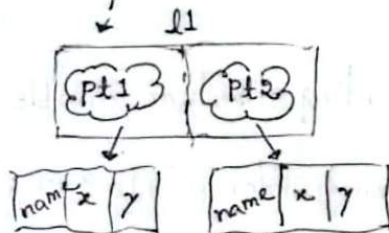
• Nesting of Structures

```

*** struct Point {
    char name[30];
    int x;
    int y; };

struct Line {
    struct Point pt1;
    struct Point pt2; };

struct Line l1;
    
```



```

*** struct x {
    int a;
    int *p; };
    
```

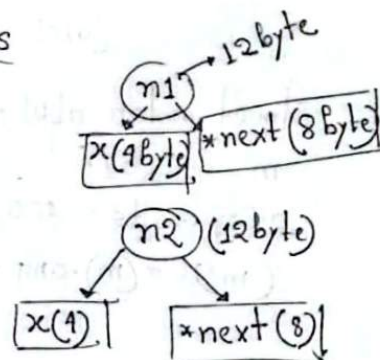
`struct x, y;`

• Self referencing structures

```

struct node {
    int x;
    struct node *next; };

struct node n1, n2;
    
```



`n1.x = 10;`

`n1.next = &n2;`

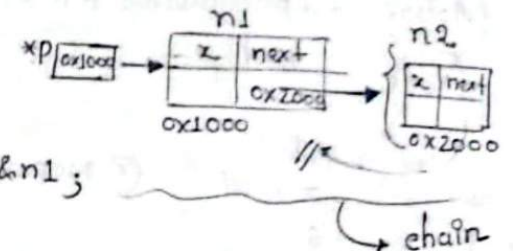
struct node type se konon variable ke point karte hain

`n2.x = 50;`

// `n2.next = &n1;`

`n2.next = Null;`

`struct node *p = &n1;`



• Linked List:

similar type data store like array

Data Structure

Linear

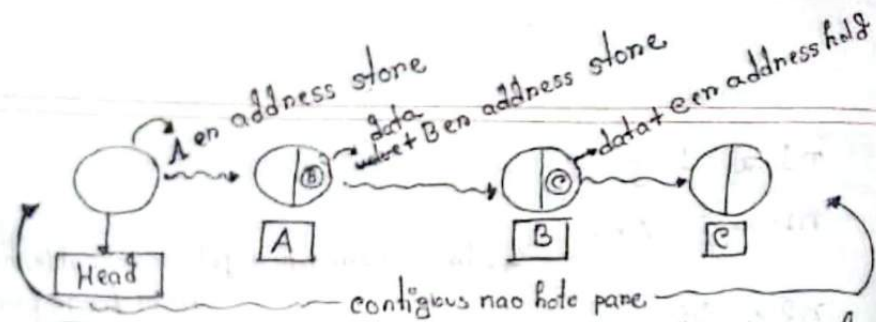
Non-Linear

sequence
thake

Linear Data Structure:

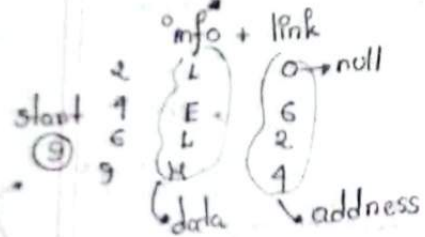
Traversal: access each data of an array on...

21/01/2025



A, B, C... : placeholders can store data and address of next placeholder.
Node

memory te space noshita hoy na. linked list



#Singly Linked List :

struct node

```
{
    int data;
    struct node *next;
};
```

each node has only one linked part (pointen ekta)

struct node *head = NULL;

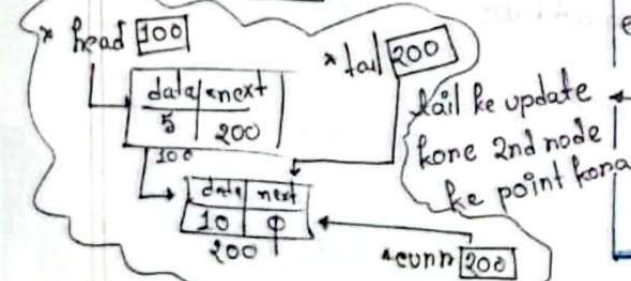
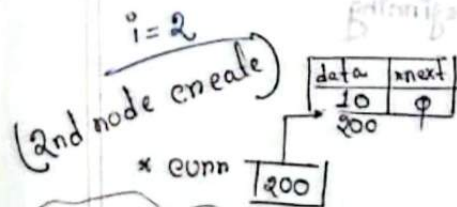
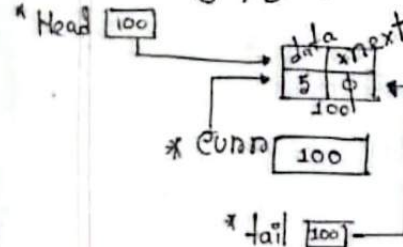
struct node *tail = NULL; → optional

struct node type
node er address store konar jonyo struct node type pointer lagbe (head)

int main() {
 create();

(loop non hobe na loop = not necessary) (number of node not necessary to define at starting)
i = 1 (1st node create)

space reserve konche
data (4 byte) + pointer (8 byte)

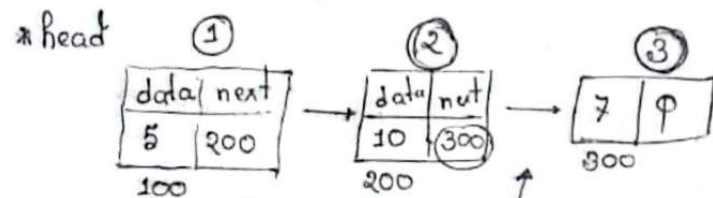


```
void create() {
    struct node *curr;
    int n, data, i;
    scanf("%d", &n); // input n
    for (i = 1; i <= n; i++) {
        scanf("%d", &data); // input data
        curr = (struct node *) malloc(
            sizeof(struct node));
        curr->val = data;
        curr->next = NULL;
        // current jai structure ke point konche
        // tar 'next' field e Null store kora hocche
    }
}
```

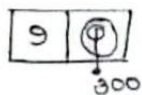
```
if (head == NULL) { head = curr;
                    tail = curr; }
else { tail->next = curr;
      tail = curr; }
```


tail jake point konche tar next field $\rightarrow$ null (tail $\rightarrow$ next: null)
 not (tail = null)

• Insertion after a particular node

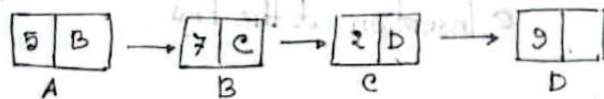


data = 9
 position = 3



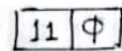
- steps {
- 1) 2-node new node ke point konbe
 - 2) newnode previous node-3 ke point konbe

* head = A



* tail

position = 3



* newnode

temp = head;

for (i = 1; i < pos; i++) {
 temp = temp;
 temp = temp $\rightarrow$ next; }
 temp1 = temp;

i = 1, temp1 = A, temp = B

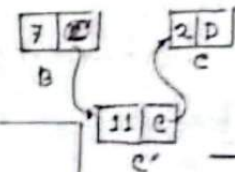
i = 2, temp1 = B, temp = C

temp1 = jar agey

newnode

temp = jar pose newnode

temp1 $\rightarrow$ next = newnode;
 newnode $\rightarrow$ next = temp;



20/01/2026

```
void insert_pos() {
    struct node *newnode, *temp, *temp1;
    create newnode;
    input : position;
    if (head == NULL) {
        head = tail = newnode;
    }
    else {
        temp = head;
        for (i = 1; i < pos; i++) {
            temp1 = temp;
            temp = temp  $\rightarrow$  next;
        }
        temp1  $\rightarrow$  next = newnode;
        newnode  $\rightarrow$  next = temp;
    }
}
```

Deletion

Deleting the first node

```
void deleteBeg(){
    struct node *temp;
    temp = head;
    head = head->next;
    free(temp); ~~~~~ memory space free
}
```

Deleting the last node

• traverse (cz last & en previous 2 ta node ke access karna hobe)

```
void deleteEnd(){
    struct node *temp, *temp1;
    if (head != NULL){
        temp = head;
        while (temp->next != NULL){
            temp1 = temp;
            temp = temp->next;
        }
        temp1->next = NULL;
        tail = temp1;
        free(temp);
    }
}
```

Deleting a particular node

temp1->next = temp->next;

Array vs Linked List

(i) fixed size vs dynamic size

↓
adjacent space

size(n) increase or decrease (m)
expensive resizing

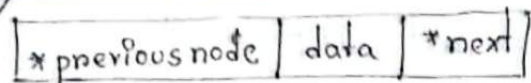
(ii) Insertion/Deletion: Inefficient (shift) vs no shifting

(iii) Random access vs serial access

(iv) Memory space: Array better if full or almost full vs no waste

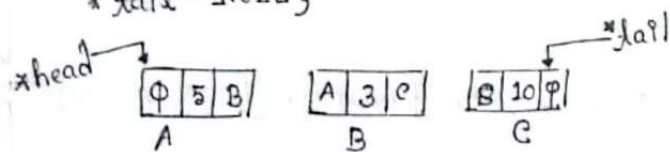
(v) Sequential access: faster vs slow

* Doubly Linked List :



```

struct node {
    int value;
    struct node *prev, *next;
}
*head = NULL;
*tail = NULL;
    
```



• Insertion

• Inserting at beginning

```

void insertBeg() {
    struct node *newnode;
    newnode = (struct node*) malloc(1 * sizeof(struct node));
    }
    
```

input data { newnode → value : scan --

newnode → prev = newnode → next = NULL;

```

if (head == NULL) { head = tail = newnode; }
    
```

```

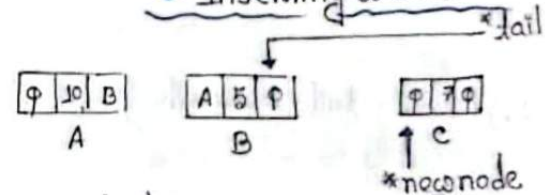
else {
    newnode → next = head;
    head → prev = newnode;
    head = newnode;
}
    
```

alter. nature

```

temp = head;
head = newnode;
temp → prev = newnode;
newnode → next = temp;
    
```

• Inserting at End



```

temp = tail;
tail = newnode;
temp → next = newnode;
newnode → prev = temp;
    
```

```

void insertEnd() {
    }
    
```

```

createNode();
    
```

```

if (head == NULL)
    head = tail = newnode;
    
```


Deletion: pop $top = 1$

```
void pop() {
    if (top == -1) stack underflow
    else { print[top];
           top--; }
}
```

Example

push(5): $top = 0$

5	7	8
0	1	2

 push(7): $top = 1$
 push(8): $top = 2$

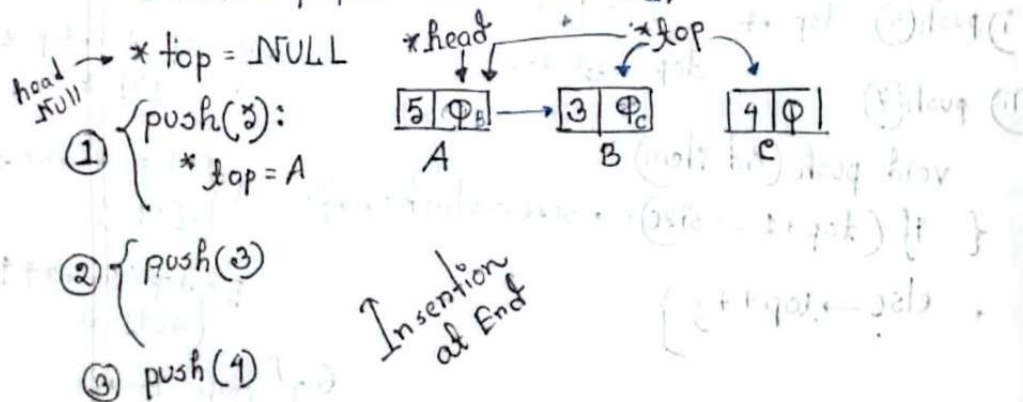
5	7	8
0		

 pop(): $top = 1$

5	2	6

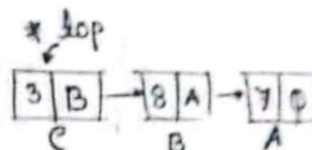
 pop(): $top = 0$
 push(2): $top = 1$
 push(6): $top = 2$

Stack implementation with singly linked list:

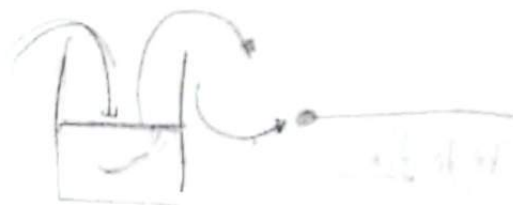


1{ POP(): traverse (delete at End)

Insertion
at Beg



pop: traverse kora lagese nh



17/02/2025

Infix Notation

$$a + b$$

$$a \oplus b$$

operator inbetween

infix

$$a + b$$

$$a + b + c$$

prefix

$$+ ab$$

$$++ abc$$

postfix

$$ab +$$

$$ab + c +$$

Prefix Notation

$$+ ab$$

Infix to Prefix

① $a + b + c$
 $\rightarrow ((a + b) + c)$ [Fully Parenthesized Expression (FPE)]

$$= ((+ab) + c)$$

$$= (+ (+ab) c)$$

$$= ++abc$$

evaluation

$$+ \quad + \quad a \quad b \quad c$$

$$\quad \quad \quad \downarrow \quad \downarrow$$

$$\quad \quad \quad (a + b)$$

$$\quad \quad \quad \downarrow$$

$$a + b + c$$

operator + (operand)
 $\rightarrow$ operation

② $a + b * c$

$$= (a + (b * c))$$

$$= (a) + (*bc)$$

$$= (+ (a) (*bc))$$

$$= +a * bc$$

Ex

$$+ a * b c$$

$\rightarrow$ 2 operands

$$(b * c) \rightarrow (i)$$

$$a + (b * c) \rightarrow (ii)$$

③ $(a + b) * c$

$$= ((a + b) * c)$$

$$= ((+ab) * c)$$

$$= * (+ab) c$$

$$= * + abc$$

prefix on postfix
 no, brackets necessary

Infix to postfix

Ex

$$a \quad b \quad c \quad *$$

$\rightarrow$ immediate post
 2 operands

$$(b * c) + a \rightarrow (ii)$$

$a + b * c$

$$= (a) + (b * c)$$

$$= (a) + (bc *)$$

$$= (a (bc *) +)$$

$$= abc * +$$

Limitation & Solution

enq(5)

enq(7)

enq(8)

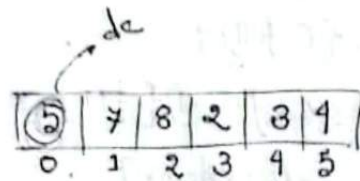
deq()

enq(2)

enq(3)

enq(4)

enq(9) → Q full but Q has element 5, size 6, limitation



const case: when array full but front empty

more efficient solution

shb element shift
time complexity ↑

Circular Array

f = -1, n = -1

c = 1 en(5) → f = 0, n = 0, Q[0] = 5

c = 2 en(7) → f = 0, n = 1, Q[1] = 7

c = 3 en(8) → f = 0, n = 2, Q[2] = 8

c = 4 de() → f = 1, n = 2

c = 3 en(1)

c = 4 en(2)

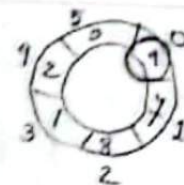
c = 5 en(3)

c = 6; en(1) → n = (n+1) % size

(6+1) % 6

= 0

Q[0] = 1, c = 6



de() → f = 2, n = 0, c = 5

de() → f = 3, n = 0, c = 4

en() → f = 3, n = 1, c = 5

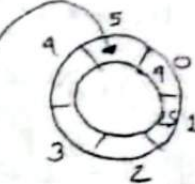
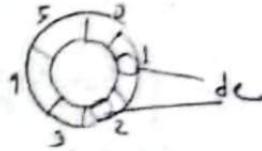
de() → f = 4, n = 1, c = 4

de() → f = 5, n = 1, c = 3

de() → f = (f+1) % size

= (5+1) % 6 = 0

f = 0, c = 2



allen
f = -1
n = -1

void en(item) {

if (c == size) { Q full }

else if (n == -1 && c == -1) { f++; c++; Q[n] = item; }

else {
n = (n+1) % size;
Q[n] = item;
c++;
}

void de() {

if (f == c == 0) { Q Empty; }

else if (f == n) or (c == 0) {
print Q[f];
f = -1, n = -1, c = -1; }

else { print Q[f];
f = (f+1) % size;
c = -1; }

}

Recursion

3) LCM (Least Common Multiplier)

① $3 \overline{) 30}$ $6 \overline{) 120}$ $30^{\min}$ divisible by both numbers
 $3 \times 1 \times 2$ $6 \times 2 \times 5$
 i) 30 -
 ii) 60 - as 12 is divisible by 6

② $3 \overline{) 45}$ $2 \overline{) 15}$
 $3 \times 2 \times 15 = 90$

6, 45
 $45 \rightarrow 6$ div. div. nh
 $15 \rightarrow 6$ div. div.
 $15 \times 2 \rightarrow 6$ div. div.

```
int main() {
    input a, b;
    if (a < b)
        res = lem(a, b);
    else
        lem(b, a);
}
```

```
int lem(int a, int b) {
    static int x = 0;
    x = x + b;
    if (x % a == 0 && x % b == 0) {
        return x;
    }
    else {
        lem(a, b);
    }
}
```

Step i) $x = 0$
 $x = 0 + 45$
 if $\rightarrow$ false
 else $\rightarrow$ lem(6, 45):
 Step ii) $x = 45 + 45 = 90$
 if (true) $\rightarrow$ return 90

4) Palindrome

```
int reverse(int n, int rev) {
    if (n == 0) {
        return rev;
    }
    else {
        return reverse(n/10, rev * 10 + n%10);
    }
}
```

Example 1

reverse(52, 0)
 i) reverse(5, 2)
 ii) reverse(0, 25)

Example 2

reverse(726, 0)
 i) reverse(72, 6)
 ii) reverse(7, 62)
 iii) reverse(0, 627)

2) GCD

① $5 \overline{) 25}$ $12 \overline{) 30}$
 $5 \overline{) 25} \rightarrow 5$ $12 \overline{) 30} \rightarrow 2$

```
int gcd(int a, int b) {
    if (b == 0) return a;
    else if (a == 0) return b;
    else return gcd(b, a % b);
}
```

i) gcd(12, 30)
 gcd(30, 12)
 ii) gcd(30, 12)
 gcd(12, 6)
 iii) gcd(12, 6)
 gcd(6, 0)
 iv) gcd(6, 0)
 return 6

Heap

i) Structural property : near complete binary tree

ii) heap property/value property:

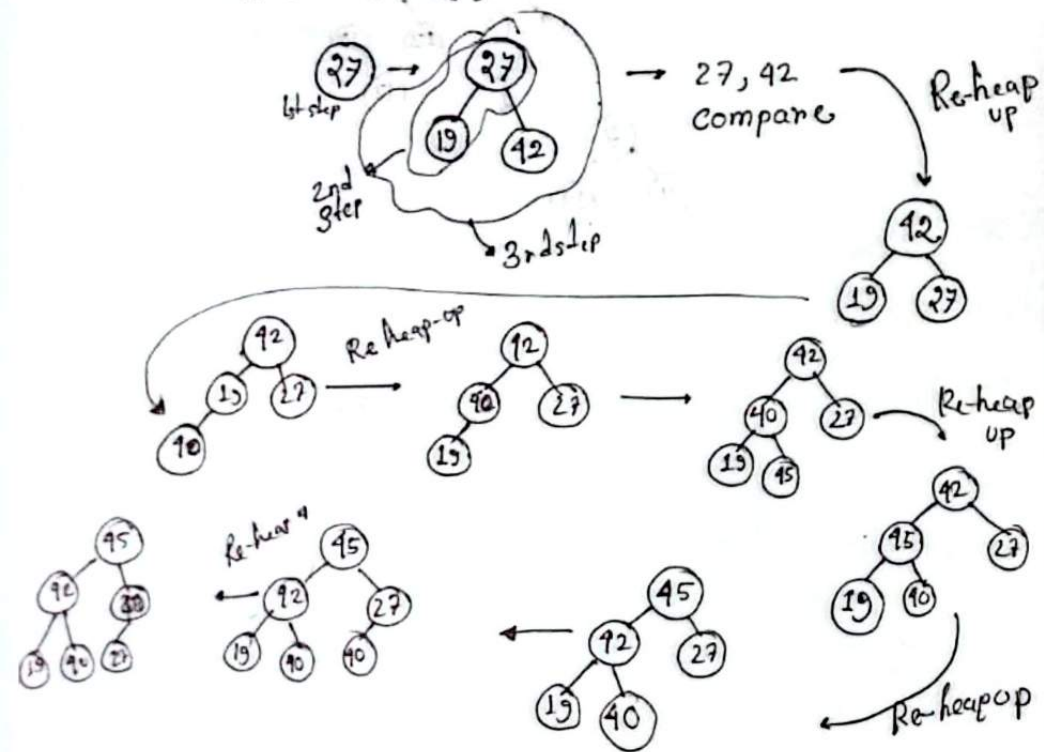
→ Max heap

→ Min heap

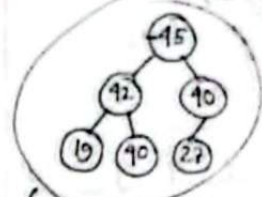
Marheap

i) insertion

27, 19, 42, 40, 15



ii) Deletion (always deleting the root)



i) 15

70 12

iii) 40

↳ last level & right most value (last node)

